

JAVA CHEAT SHEET — TESD 1800: GUI + JavaFX

BASIC PROGRAM STRUCTURE

Every JavaFX app starts with a class that **extends Application**.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;

public class MyApp extends Application {
    @Override
    public void start(Stage stage) {
        Pane pane = new Pane();           // layout container
        Scene scene = new Scene(pane, 400, 200);
        stage.setTitle("My JavaFX App");
        stage.setScene(scene);
        stage.show();                     // must call this
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

COMMON LAYOUT PANES

Use layouts to position UI controls.

```
HBox h = new HBox(10); // horizontal, 10px gap
VBox v = new VBox(10); // vertical, 10px gap
BorderPane b = new BorderPane(); // top, bottom, left, right, center
GridPane g = new GridPane(); // rows + columns
StackPane s = new StackPane(); // centers elements
Pane p = new Pane(); // absolute positions
```

Add nodes:

```
h.getChildren().addAll(btn1, btn2);  
b.setCenter(label);  
v.getChildren().add(textField);
```

BASIC UI CONTROLS

LABEL

Displays text or an image.

```
Label lbl = new Label("Programming is fun");
```

You can combine text + graphic:

```
lbl.setGraphic(new ImageView("image/us.gif"));
```

BUTTON

Triggers an action.

```
Button bt = new Button("Click Me");  
bt.setOnAction(e -> System.out.println("Button clicked!"));
```

CHECKBOX

Allows multiple independent choices.

```
CheckBox chk = new CheckBox("Bold");  
chk.setOnAction(e -> System.out.println(chk.isSelected()));
```

RADIOBUTTON

Select *one* option from a group.

```
ToggleGroup group = new ToggleGroup();  
RadioButton rbRed = new RadioButton("Red");  
RadioButton rbBlue = new RadioButton("Blue");
```

```
rbRed.setToggleGroup(group);  
rbBlue.setToggleGroup(group);
```

TEXTFIELD

Single-line text input.

```
TextField tf = new TextField();  
tf.setOnAction(e -> System.out.println(tf.getText()));  
tf.setEditable(true);    // allow typing  
tf.setAlignment(Pos.CENTER_RIGHT);
```

TEXTAREA

Multi-line text input.

```
TextArea ta = new TextArea("Type here...");  
ta.setWrapText(true);  
ta.setEditable(true);
```

COMBOBOX + LISTVIEW

COMBOBOX (Dropdown)

```
ComboBox<String> cbo = new ComboBox<>();  
cbo.getItems().addAll("Red", "Blue", "Green");  
cbo.setValue("Red");  
cbo.setOnAction(e -> System.out.println(cbo.getValue()));
```

LISTVIEW (List of items)

```
ListView<String> lv = new ListView<>();  
lv.getItems().addAll("Dog", "Cat", "Fish");  
lv.getSelectionModel().setSelectionMode(SelectionMode.MULTIPLE);  
lv.getSelectionModel().selectedItemProperty().addListener(  
    ov -> System.out.println(lv.getSelectionModel().getSelectedItems())  
);
```

SCROLLBAR + SLIDER

SCROLLBAR

```
ScrollBar sb = new ScrollBar();  
sb.valueProperty().addListener(ov ->  
    System.out.println("Value: " + sb.getValue())  
);
```

SLIDER

```
Slider sl = new Slider();  
sl.setShowTickMarks(true);  
sl.setShowTickLabels(true);  
sl.valueProperty().addListener(ov ->  
    System.out.println(sl.getValue())  
);
```

COLOR AND STYLE

Change text color:

```
text.setFill(Color.RED);
```

Add a border:

```
pane.setStyle("-fx-border-color: green; -fx-border-width: 2;");
```

Set font:

```
text.setFont(Font.font("Times New Roman", 20));
```

TIMELINE (ANIMATION + TIMER)

Perfect for countdowns or smooth movement.

```
Timeline timeline = new Timeline(  
    new KeyFrame(Duration.seconds(1), e -> countDown())
```

```
);  
timeline.setCycleCount(Timeline.INDEFINITE);  
timeline.play();
```

Stop or pause:

```
timeline.stop();  
timeline.pause();
```

MEDIA + SOUND

Play audio or video:

```
Media media = new Media("file:/C:/path/to/Rooster.mp3");  
MediaPlayer player = new MediaPlayer(media);  
player.setCycleCount(MediaPlayer.INDEFINITE); // loop  
player.play();
```

Display video:

```
MediaView view = new MediaView(player);  
pane.getChildren().add(view);
```

EVENT HANDLING

Event = something happens (click, type, etc.)

Lambda style:

```
btn.setOnAction(e -> handleClick());
```

Traditional:

```
btn.setOnAction(new EventHandler<ActionEvent>() {  
    public void handle(ActionEvent e) {  
        handleClick();  
    }  
});
```

TICTACTOE LOGIC RECAP

- Use a 3x3 `GridPane`
- Each cell is a custom `Pane` subclass that listens for mouse clicks
- Use `whoseTurn` char ('X' or 'O')
- Check for win with rows, columns, and diagonals
- When done, update label with result

COMMON JAVAFX IMPORTS

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.*;
import javafx.scene.control.*;
import javafx.scene.text.*;
import javafx.scene.paint.Color;
import javafx.scene.image.*;
import javafx.event.*;
import javafx.animation.*;
import javafx.util.Duration;
```

QUICK TROUBLESHOOT

- Always call `launch(args)` from `main()`.
- JavaFX files must be in a **JavaFX project setup** (with module path set).
- Images and sound need full paths or must be in the project `/resources/` folder.

- Missing `@Override` means your method name might be wrong (check `start()` spelling).
 - Don't forget to import `javafx.*` packages.
-

MINI EXAMPLES

Move Text Left/Right:

```
btLeft.setOnAction(e -> text.setX(text.getX() - 10));  
btRight.setOnAction(e -> text.setX(text.getX() + 10));
```

Countdown Timer:

```
int[] time = {10};  
Timeline t = new Timeline(  
    new KeyFrame(Duration.seconds(1), e -> {  
        time[0]--;  
        label.setText(time[0] + " sec");  
        if (time[0] <= 0) player.play();  
    })  
);  
t.setCycleCount(10);  
t.play();
```

Change color with RadioButtons:

```
rbRed.setOnAction(e -> text.setFill(Color.RED));  
rbBlue.setOnAction(e -> text.setFill(Color.BLUE));
```

STYLE SHORTCUTS

```
node.setStyle("-fx-font-size: 16; -fx-text-fill: orange;");  
pane.setAlignment(Pos.CENTER);
```

REMEMBER:

- All JavaFX apps start with **Application** → **Stage** → **Scene** → **Pane**.
- UI controls are nodes; panes hold nodes.
- Every control has properties you can bind or listen to.
- Use layout managers (HBox, VBox, etc.) — don't hard-code positions unless needed.
- Keep logic separate from layout — clean, readable code always wins.

Would you like me to format this next into a **printable PDF mousepad-style layout** (dark background with gold or white text like your screenshots)?